Provider Network Access and Cost Planner, Part 3: Neural and Probabilistic Models for High-Cost Prediction and Document Reading

David Satizabal

Miami Dade College

CAI4510C: Machine Intelligence

Project Part 3: Neural Models Integration

Dr. Martins Idahosa

November 9, 2026

Provider Network Access and Cost Planner, Part 3: Neural and Probabilistic Models for High-Cost Prediction and Document Reading

In Part 2 I built a pipeline that flags health plan members who are likely to be high-cost next quarter. Its champion was a logistic regression with balanced class weights. The flag feeds my AI course's planner, which seats members with doctors and spreads the heaviest patients. For Part 3 I tested whether a neural network beats that champion, whether a network built for images reads the scanned paperwork from my AI Part 2 work better, and whether a probabilistic model can explain expensive stretches in a way a care manager could check. All data is synthetic and seeded (seed 42). No real members, providers, or claims are used.

Problem and Data Snapshot

I used my term project's data instead of the provided toy sets so the models plug into the same planner. The tabular data has 31,563 member-month rows for 1,500 members from January 2025 to September 2026. I removed 63 duplicate rows. A row is high-cost when the member's cost over the next three months is at least $3,904.47, the top 10% of training rows. The split is by time. Training covers January to September 2025 (13,500 rows), then there is a three-month gap, and testing covers January and February 2026 (3,000 rows, 305 high-cost, 10.2%).

For early stopping I held out 20% of the training **members**, not rows, so no member's months land on both sides. That left 10,800 fit rows (1,035 high-cost) and 2,700 validation rows (315 high-cost). Each row has 17 features after preprocessing: nine scaled numbers (age, conditions, miles to a primary care doctor, visits, ER visits, admissions, and three cost totals) and one-hot sex, county, and plan type (missing filled as Unknown), fit on the fit rows only.

The vision data is 1,500 synthetic 64 by 64 grayscale scans of three document types: claim forms, referrals, and enrollment forms. There are 840 training, 180 validation, and 180 test

images, plus 300 messy scans with tilt, blur, noise, and uneven lighting that no model ever trains on.

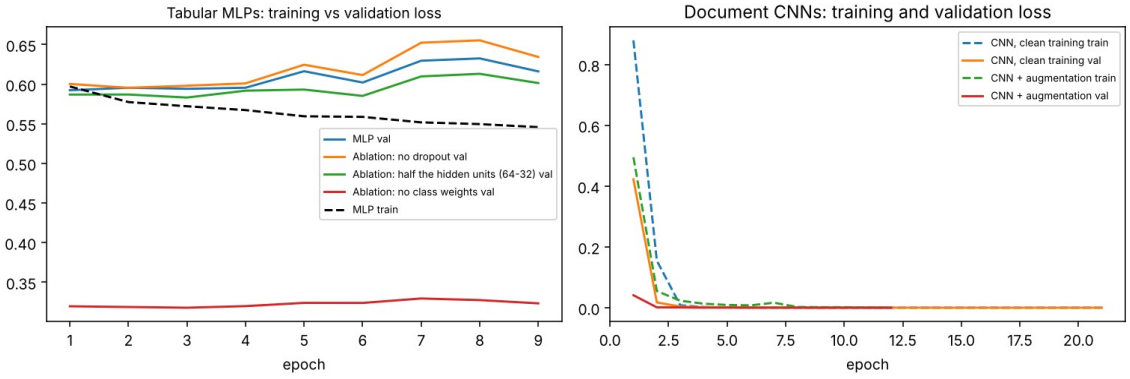
Architectures and Hyperparameters

Multilayer Perceptron

The MLP follows the assignment's spec: 17 inputs, a dense layer of 128 with ReLU and 20% dropout, a dense layer of 64 with ReLU, and two outputs. That is 10,690 parameters. I trained it with Adam (learning rate 0.001, weight decay 0.0001), batch size 64, up to 50 epochs, and early stopping on validation ROC-AUC with a patience of 8. The loss is cross-entropy with balanced class weights, because only 10% of rows are high-cost and Part 2 showed that an unweighted model barely flags anyone. Validation ROC-AUC peaked after the first epoch (0.760). Training stopped at epoch 9 and kept the epoch-1 weights. Figure 1 shows why: training loss kept falling (0.597 to 0.546) while validation loss rose, so the extra epochs were memorizing training members.

Figure 1

Training and Validation Loss for the Tabular MLPs and the Document CNNs



Note. Left: the MLP's validation loss rises after epoch 1 while training loss falls. Right: both CNNs reach near-zero loss within a few epochs. Full logs are in outputs/training_logs.csv.

Convolutional Neural Network

The CNN also follows the spec: two blocks of a 3 by 3 convolution (8, then 16 filters), ReLU, and 2 by 2 max pooling, then a dense layer of 32 and three outputs. That is 132,451 parameters, almost all in the first dense layer. Pixels are scaled so ink is 1 and paper is 0. I trained it with Adam (learning rate 0.001), batch size 32, up to 50 epochs, and early stopping on validation loss with a patience of 6. I trained one version on the 840 clean images and one on 2,520 (each image plus two copies with tilt, shift, blur, noise, and a lighting gradient).

Architecture Rationale

I started with the MLP because it is the smallest step away from Part 2. It uses the same features, split, and class weights, so any gain has to come from the model's ability to bend, meaning to learn interactions a straight line cannot.

I picked the CNN track over the LSTM because of what my data looks like. A document's type is in its layout: a grid of boxes, a signature line, a header bar. A CNN's built-in assumption, its inductive bias, is spatial locality. A small filter looks at a few neighboring pixels at a time and is reused across the whole page, so a box edge is recognized wherever it lands, and pooling adds some tolerance for shifts. My AI Part 2 softmax treats every pixel as its own feature, so a checkbox shifted by three pixels looks like new evidence. An LSTM's bias, sequence memory, fits monthly bills, but the summary features already carry most of each member's short history, so I covered sequences with the readable HMM instead.

Spatial locality clearly helped when practice data was clean. Trained on clean scans only, the CNN read 90.3% of messy scans, against 78.0% for the softmax (Table 2). Augmentation closed that gap by showing the softmax shifted and blurred copies, which lifted it to 99.7%. The bias is a substitute for data: it helps most when the training set doesn't already show the variety.

On the tabular side no bias helped. A row of 17 features has no spatial or time order for a network to exploit (LeCun et al., 2015).

Metrics and Calibration

Table 1 compares every tabular model on the test months using the assignment's template. "Project 2 best" is refit on the same 80% of training members as the MLP so the comparison is fair. Its test ROC-AUC is 0.755 here, against 0.752 in Part 2.

Table 1

Comparison Against the Project 2 Champion (High-Cost Flag, Test Months)

Model	Features	Val Acc	F1 (macro)	ROC-AUC	Recall	ECE	Params	Train time	Notes
Project 2 best: logistic regression, balanced	Tabular (17)	.713	.565	.755	.672	.019	18	< 0.1 s	Traditional baseline
MLP, class weights	Tabular	.732	.565	.755	.653	.033	10,690	1.3 s	Kept epoch 1
MLP + MC Dropout (20 passes)	Tabular	.733	.564	.755	.636	.036	10,690	1.3 s	Variance = uncertainty
Gaussian Naive Bayes	Tabular	.844	.616	.736	.298	.108	68	< 0.1 s	Interpretable
HMM (2 states)	Monthly cost bands	.829	.590	.673	.253	.010*	14	1.5 s	Readable tables

Note. Recall is the share of the 305 high-cost test rows each model caught. ECE follows the assignment's definition (confidence of the chosen answer). *The HMM almost always answers "no," so its low ECE says little; Figure 2 checks the chance of high-cost directly.

The MLP ties the champion. Its ROC-AUC is 0.7553 against 0.7554, and its recall is slightly lower (0.653 against 0.672) with about 600 times the parameters. Its test confusion matrix (outputs/mlp_confusion_test.png) shows 199 of 305 high-cost rows caught, 106 missed, and 763 false alarms. The pattern in this data is close to additive (older, more conditions, more cost last year), so a model that can bend has nothing extra to find. Deep networks rarely beat simpler models on ordinary tabular data (Grinsztajn et al., 2022). Naive Bayes looks better on accuracy only because it flags fewer members and catches 30%, the accuracy paradox again. **The simpler model stays champion.**

Table 2 shows the document results. With clean training, the CNN wins on messy scans by 12 points. With augmentation, the simple softmax edges ahead (99.7% against 97.0%). Augmentation also cut the CNN's calibration error on messy scans from 0.067 to 0.023.

Table 2

Document Type Accuracy, Clean Test Scans and Messy Scans

Model	Training images	Clean test	Messy scans	Messy ECE	Params
AI Part 2 softmax, clean training	840	1.000	0.780		12,291
CNN, clean training	840	1.000	0.903	0.067	132,451
AI Part 2 softmax + augmentation	2,520	1.000	0.997		12,291
CNN + augmentation	2,520	1.000	0.970	0.023	132,451

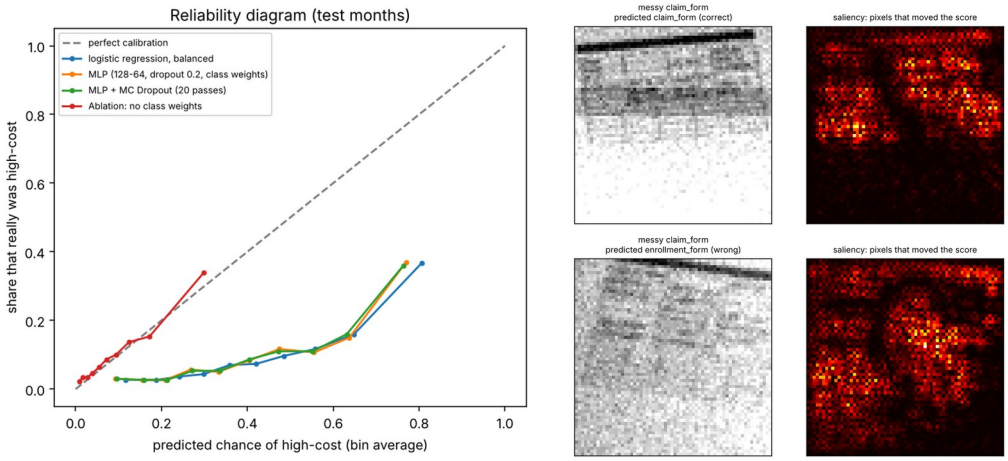
Note. Messy scans (n = 300) are never used in training. The CNN's confusion matrix on the messy scans is in outputs/cnn_confusion_messy.png.

Calibration

ECE as the assignment defines it checks the confidence of whichever answer the model picked. The balanced logistic regression scores a low 0.019 because most of its answers are a confident and correct "no." The reliability diagram (Figure 2, left) checks the chance of high-cost directly, and it tells a different story. When the class-weighted models say about 60%, only about 15% of those members were really high-cost. When they say about 80%, about 37% were. Class weights tell the model that missing a high-cost member costs about nine times more than a false alarm, and that pushes every probability up. The unweighted MLP sits near the diagonal but catches only 3.9% of high-cost members (Guo et al., 2017, describe this kind of miscalibration).

Figure 2

Reliability Diagram for the Chance of High-Cost (Left) and CNN Saliency Maps for Two Messy Claim Forms (Right)



Note. Left: quantile bins; points below the dashed line mean the model overstates risk. Right: top row correct, bottom row wrong (CNN, clean training); brighter pixels changed the class score more.

Uncertainty Discussion

Calibration changed three decisions. First, I stopped treating 0.5 as a yes. At 0.5, the logistic regression would flag 195 of the 600 planner members, more than three times the real high-cost share, so the hand-off ranks members and flags the riskiest 10% (60 members). Second, the scores are used only to rank members for outreach. They are never quoted to a care manager as a percentage. Third, the HMM's yes line is set the same way (the riskiest 10% of validation rows, P of 0.147 or more). Quoting real percentages would first need temperature scaling on the validation set (Guo et al., 2017).

For MC Dropout I left dropout on at prediction time and ran each member through the network 20 times (Gal & Ghahramani, 2016). The average barely changed the ranking (ROC-AUC 0.7548), but the spread shows which members the model is unsure about. Two examples follow. Member M1150 (February 2026) is 56, with one chronic condition, two ER visits, and $60,270 in costs over the prior 12 months. The average prediction was 0.574 with a standard deviation of 0.167, and the member's next quarter was $1,841, not high-cost. Member M1455 is

40, with one condition, one ER visit, and $98,114 in prior-year costs. The average was 0.337 with a standard deviation of 0.139, and the next quarter was $143. Both have one huge past bill and few other warning signs, and the 20 runs disagree on whether last year was a one-time event. In the integrated pipeline I built afterward, the 60 planner members with the highest spread get a care-team review note. Only 7 of them are also in the high-cost 60, so the spread surfaces different people than the score does.

Visual Explanations

For the CNN I used gradient saliency, which measures how much each pixel would change the winning class's score (Simonyan et al., 2014). Figure 2 (right) shows two messy claim forms. On the one the CNN got right, the heat sits on the grid of boxes in the top half of the page, which is the part that makes a claim form a claim form. On the one it called an enrollment form, the scan is faded and grainy, and the heat is spread across the whole page, including the noise. The fixes are augmentation (90.3% to 97.0%) and routing unsure scans to a person.

Probabilistic Model Results

The HMM has two hidden states each month: normal, or in a high-cost spell (the last three months' cost is in the training top 10%, at least $4,034). What it observes each month is the bill size in four bands: under $100, $100 to $999, $1,000 to $4,999, and $5,000 or more. I counted the start, transition, and emission tables from the training months and added 1 to every count so no probability is ever exactly zero. A normal member has a 3.2% chance of starting a spell next month, and a member in a spell has a 68.6% chance of staying in it. In a spell, 22.3% of months are $5,000 or more, against 0.01% in normal months. Viterbi decoding gives the most likely state path. For member M0558, the bands from January 2025 to February 2026 were 1, 1,

0, 0, 0, 3, 0, 1, 0, 0, 3, 2, 1, 0. Viterbi places a spell only in June 2025 and in November and December 2025, so a single expensive month followed by quiet ones is read as a blip, while two heavy months in a row are read as a stretch.

The HMM predicts worse than the tabular models (ROC-AUC 0.673, recall 0.253) because it sees only bill sizes, not age or conditions. One honest limit: the hidden state was computed from known cost so the tables could be counted directly. A true HMM would learn states it never sees with Baum-Welch (Rabiner, 1989). Gaussian Naive Bayes ranks better (0.736) with 68 parameters, but it has the worst calibration (ECE 0.108). It assumes features are independent, so it counts the three overlapping cost totals as three separate pieces of evidence.

Probabilistic Justification

NB and HMM would be the better choice in three situations. The first is small data. They only need counts, and generative models reach their best error with far fewer examples than discriminative ones (Ng & Jordan, 2002). Mine already memorized after one pass over 10,800 rows. The second is drift. In my data, Pinecrest Medical Group leaves the network in June 2026. The HMM's tables can be recounted on the new months in seconds, and I can see which number moved, such as the chance of starting a spell. The MLP would hide the change in 10,690 weights. The third is when someone needs a posterior they can read. A care manager can be told "members in an expensive stretch stay in it about 69% of the time," and an auditor can check that number against the counts. Deep networks earn their place with lots of data or raw inputs like images.

Ablations and Error Analysis

Table 3 removes one piece of the MLP at a time. Dropout and network size barely matter. Halving the hidden units cut two thirds of the parameters and changed almost nothing. Class

weights are the single most important choice. Without them, accuracy rises to 90.2% and recall collapses from 65% to 4%: the model catches 12 of 305 high-cost members.

Table 3

MLP Ablations (Test Months, Change Against the Full Model)

Version	Params	Recall	Change	ROC-AUC	Change	Accuracy
Full MLP (128-64, dropout, class weights)	10,690	.653		.755		.710
No dropout	10,690	.633	-.020	.754	-.001	.720
Half the hidden units (64-32)	3,298	.639	-.013	.755	.000	.730
No class weights	10,690	.039	-.613	.746	-.009	.902

Note. Recall is for the high-cost class. Every version kept its epoch-1 weights.

I sorted all 869 of the MLP's test mistakes into four causes, using each member's prior-year cost against the training median and 90th percentile, and I inspected eight of them (two per cause, listed in outputs/metrics.json). The largest group is **class overlap**: 589 false alarms on members whose profile looks risky but who stayed low. Next are 174 false alarms on members who were expensive last year and quiet next quarter, which are one-time events. The misses split in two. Fifty-five members had real warning signs but scored just under the line (median 0.42), such as a 48-year-old with three conditions scored 0.481 who cost $6,628. That is a threshold problem. Fifty-one came out of nowhere: a 29-year-old with one condition and $2,197 in prior-year costs, scored 0.117, then cost $6,125. Nothing in the features points to that, so it is a feature problem no model can fix.

The per-group check shows a real skew. The flag catches 91.4% of high-cost members aged 65 and over, 33.7% of those aged 45 to 64, and **none** of the high-cost members aged 18 to 44 (734 test rows, 4.5% high-cost, 1.1% flagged). Medicaid members get 25% recall against 91.8% for Medicare Advantage. The two counties are close (63.1% and 66.5%). Part of the gap is real (seniors are high-cost 19.4% of the time against 4.5%), but zero recall means young members never get an outreach call.

Ethical and Responsible AI Reflection

Overconfidence and fairness. The age and Medicaid gaps above mean outreach goes mostly to older and Medicare Advantage members. The weighted models overstate risk about two to three times. If a score were quoted as a percentage, a care manager would think a member had a 60% chance of a costly quarter when the real rate is about 15%.

Dataset artifacts. I wrote the data generator, so the models learn my rules, not real-world truth, and the synthetic scans are cleaner and more uniform than real paperwork. The target is also a risk in itself. A widely used health algorithm that predicted cost as a stand-in for need underestimated the needs of Black patients, because less money had been spent on their care (Obermeyer et al., 2019). My target has the same weakness: high cost is not the same as high need.

Mitigations. First, the flag is used for outreach calls and doctor placement only, never to deny care or set prices. Second, confidence thresholds: scans below 70% confidence go to a person (in AI Part 2 that auto-routed 99.3% of messy scans with zero wrong routes), and members with high MC Dropout spread go to human review. Third, a human in the loop for the known blind spot: separate yes lines by age group, more signals than age, and review of young members with sudden ER use. Fourth, calibrate before showing any probability, and track recall by group every month and after drift.

Conclusion

The neural network tied the Part 2 champion on the tabular flag, so the simpler model stays champion. The CNN's spatial bias helped most with clean training data, class weights mattered more than any architecture, and calibration showed the scores should rank members, not be quoted.

References

Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning* (Vol. 48, pp. 1050–1059). PMLR.

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems* (Vol. 35, pp. 507–520). Curran Associates.

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (Vol. 70, pp. 1321–1330). PMLR.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*(7553), 436–444. https://doi.org/10.1038/nature14539

Ng, A. Y., & Jordan, M. I. (2002). On discriminative vs. generative classifiers: A comparison of logistic regression and naive Bayes. In *Advances in Neural Information Processing Systems* (Vol. 14, pp. 841–848). MIT Press.

Obermeyer, Z., Powers, B., Vogeli, C., & Mullainathan, S. (2019). Dissecting racial bias in an algorithm used to manage the health of populations. *Science*, *366*(6464), 447–453. https://doi.org/10.1126/science.aax2342

Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, *77*(2), 257–286. https://doi.org/10.1109/5.18626

Simonyan, K., Vedaldi, A., & Zisserman, A. (2014). *Deep inside convolutional networks: Visualising image classification models and saliency maps* (arXiv:1312.6034). arXiv. https://doi.org/10.48550/arXiv.1312.6034